Testing Small Vision Models for On-Device Eye Tracking

Project note from the EyeCoD line of work

Lintong Han | lintongbarry.github.io

Georgia Institute of Technology, with Haoran You, 2023–2024. Not peer reviewed.

1 The project in brief

Virtual and augmented reality headsets need to know where the wearer is looking. Eye tracking makes it possible, for example, to render only the region the eye is focused on in full detail. Doing it well is demanding: the tracker has to run hundreds of times per second on a small, battery-powered device, close to the user's eye.

EyeCoD, published at ISCA 2022 by Haoran You and colleagues [1], tackles this with three ideas. It replaces the usual lens camera with FlatCam, a thin lensless camera, so the camera can sit right next to its processor. It uses a "predict-then-focus" approach: first find the eye region, then estimate gaze only on that small crop instead of the whole image. And it pairs these with a dedicated chip designed for exactly these two steps. On measured hardware the authors report large speedups over CPUs, GPUs and an earlier eye-tracking processor while keeping tracking accuracy close to a lens-based setup [1]. The work was later selected for IEEE Micro's Top Picks [2].

I did not take part in writing the EyeCoD paper.

2 What I did: testing

My role was testing for a follow-up question in the same line of work: once we know where someone is looking, can a small model cut out the object they are looking at, quickly enough to run on the device?

Recent "promptable" segmentation models can do this from a single point: give them a location in the image and they return a mask of the object there [3]. The gaze point is a natural prompt. The catch is size, since the original models are far too large for a headset. Smaller variants exist, such as MobileSAM [4].

- I ran compact segmentation models side by side on the same images and gaze points, comparing how good the masks were against how large and how slow each model was.
- I looked at the cases where each one failed, such as small objects, cluttered scenes and gaze points near an object's edge.
- The comparison led the team to choose MobileSAM as the best balance between mask quality and model size.
- I also tested a simple scheduling idea: when the gaze is steady, keep the previous mask instead of recomputing it, and only update when the gaze moves.

3 What I learned

- **The device decides the model.** On a headset, a model that is slightly less accurate but fits in memory and runs in time is worth more than a better one that does not.

- **Measure the whole pipeline.** Speed on paper and speed on the device can differ; it is the end-to-end delay the wearer feels.
- **Look at failures, not only averages.** The failure cases told us more about which model to pick than a single score did.

This was my first research experience, and it shaped how I approach later projects: treat compute and latency as part of the problem from the start.

References

[1] H. You, C. Wan, Y. Zhao, Z. Yu, Y. Fu, J. Yuan, S. Wu, S. Zhang, Y. Zhang, C. Li, V. Boominathan, A. Veeraraghavan, Z. Li, Y. Lin. EyeCoD: eye tracking system acceleration via FlatCam-based algorithm and accelerator co-design. *ISCA*, 2022. arXiv:2206.00877.

[2] H. You et al. EyeCoD: eye tracking system acceleration via FlatCam-based algorithm and hardware co-design. *IEEE Micro* 43(4):88–97, 2023.

[3] A. Kirillov et al. Segment anything. *ICCV*, 2023.

[4] C. Zhang et al. Faster segment anything: towards lightweight SAM for mobile applications. arXiv:2306.14289, 2023.